

SYSTEMATIC LITERATURE REVIEW

Semantic Caching for LLM Inference

A Systematic Literature Review — 87 records screened to 66 papers

Kaniz Reza Mithila

ID: 2022-3-60-052

Khaled Mahmood Sifat

ID: 2022-3-60-217

Redwan Ahmed

ID: 2022-3-60-155

◀ The Problem

LLM inference recomputes everything, every time

- LLM inference is computationally expensive — every query recomputes from scratch
- Semantic caching is the field's proposed fix: reuse a stored response for a similar new query
- It is frequently marketed as a green-computing technique

Repeated Query

Cache Hit

~~Recompute from
Scratch~~

*Semantic caching decides reuse by meaning,
not exact text match*

◀ THE GAP & OUR CONTRIBUTION

No review has consolidated this literature — until now

- No published review consolidates this fast-growing literature
- This review: systematic search, five databases, three rounds, 87 candidate records

87

records identified

84

screened

66

papers included

5

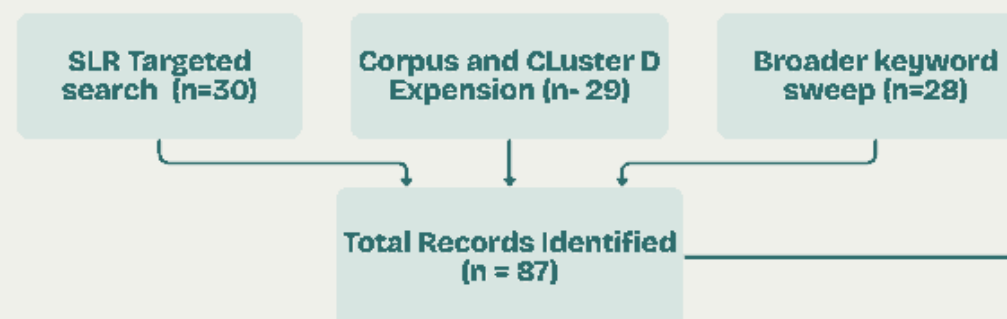
clusters

We also found something the field has not measured.

METHODOLOGY

Three
search
rounds,
full-text
screening,
five cluster

IDENTIFICATION



Records removed before screening (n = 3)
cited in Introduction (n = 3)

SCREENING

Records screened
(n = 84)

Records excluded
(n = 0)

Reports sought for retrieval
(n = 84)

Reports not retrieved
(n = 0) 100% retrieved

Reports assessed for
eligibility (n = 84)

Reports Excluded (n=18)

Wrong domain (n = 7)

Duplicate paper (n = 1)

No solid contribution (n = 5)

Wrong mechanism (n = 5)

Included

Studies included in review (n = 66)

Cluster A: Core LLM
Semantic Caching (26)

Cluster B: Adaptive
Threshold / Eviction /
Verification (12)

Cluster C: Security &
Reliability of Caching (5)

Cluster E: KV-cache /
Prefix-level, contrast
evidence (11)

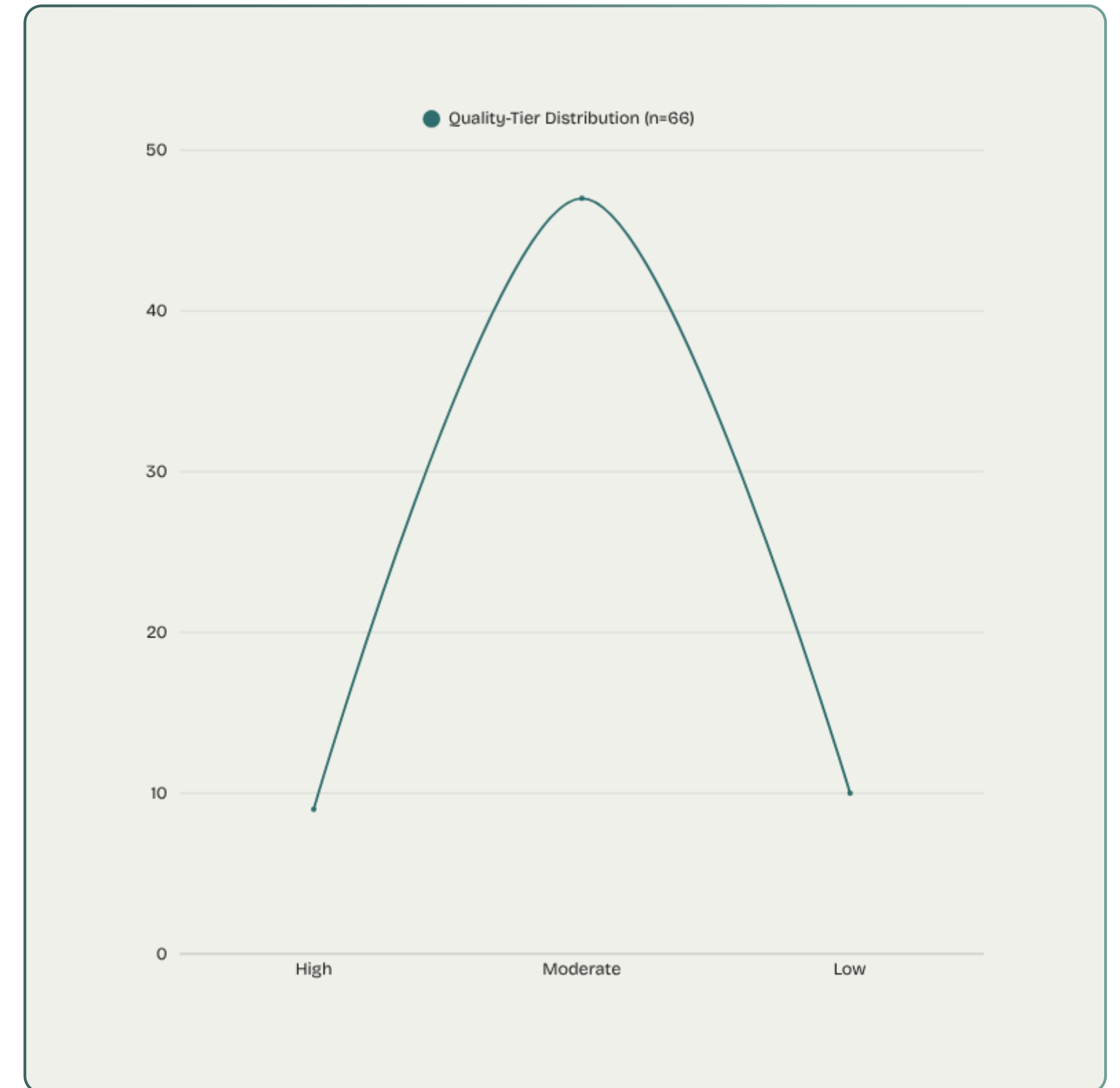
Cluster D: Agentic /
Domain-Specific
Caching (12)

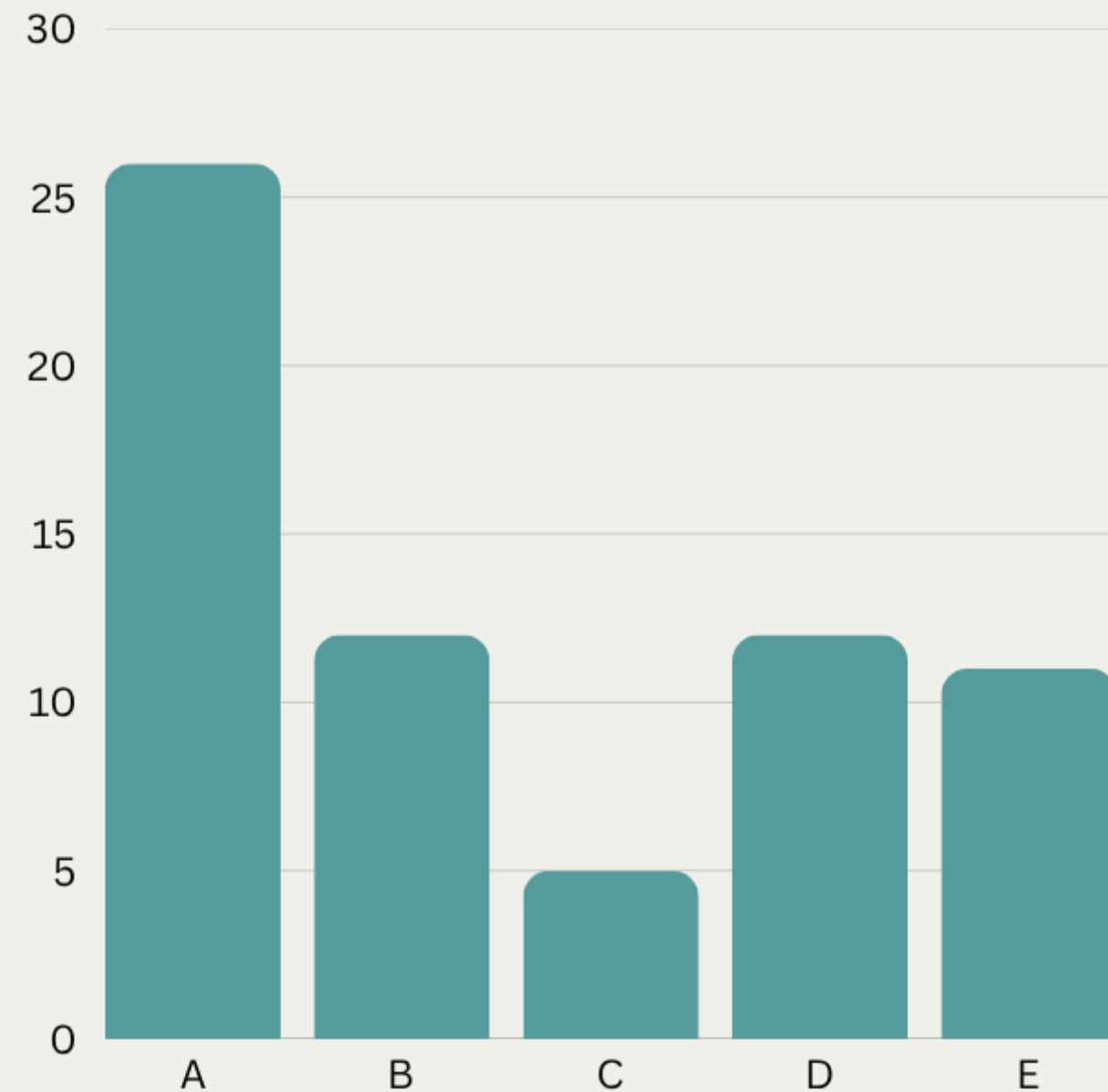
QUALITY ASSESSMENT

Quality matters as much as inclusion

- A four-dimension rubric scored every included paper: venue rigor, evaluation depth, reproducibility, limitation transparency

*Low tier stays in the corpus.
It just carries less weight.*





Five Clusters

A-Core LLM Semantic Caching

B -Adaptive Threshold / Eviction / Verification

C -Security & Reliability of Caching

D -Agentic / Domain-Specific Caching

E -KV-Cache / Prefix-Level Caching
(contrast evidence — not core)

Cluster E is kept separate on purpose — it reuses internal model state within a single generation, a structurally different mechanism from cross-query semantic caching. It is retained as contrast evidence, not core evidence.

Five Patterns That Recur

P1

The similarity threshold
is unreliable

P2

Caching stays blind to
context

P3

Each fix improves one
thing and leaves the
next

P4

Reliability fixes add
unmeasured cost

P5

Cached results going stale
is understudied

THE CENTRAL FINDING

Not one paper in this corpus measures the energy cost of semantic caching against the inference it replaces

18 papers report comparable results. **0** report an energy figure.

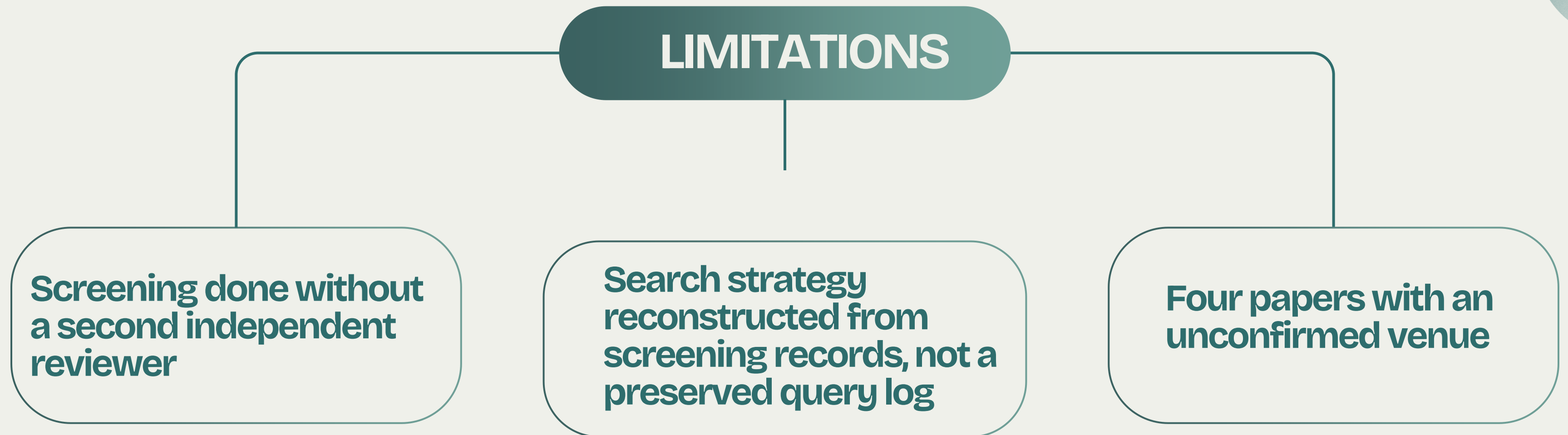
OPEN QUESTIONS FOR THE FIELD

Three Open Questions

- 1 Under what workload and threshold conditions does query-level semantic caching produce a net energy saving, once embedding generation, vector search, and verification overhead are included in the accounting — not only the inference calls the cache avoids?
- 2 Is there a minimum cache hit rate below which semantic caching costs more energy than always performing direct inference?
- 3 Does a high cache hit rate reliably imply high net energy savings, or can the two diverge?

◀ LIMITATIONS

We want to be upfront about our own limits





CONCLUSION

What we built — A 66-paper corpus across five clusters, with a four-dimension quality rubric applied to every paper

What we found — Five recurring patterns — no single paper states any of them alone

What we're proposing — Three open questions the field has left unanswered on energy cost

**Not one paper in this corpus measures whether semantic caching actually
saves energy.**

That's the gap we're leaving the field with.



Thank

You

